CLAUDE.MD — CLAUDE CODE 작업 지침서

AgentOps (가칭 · 정식명: Spanlens)

Claude Code가 이 프로젝트에서 안전하고 일관되게 작업하기 위한 지침서

v1.0	2026.04	Next.js 14 + Hono + Supabase	SaaS (클라우드)
버전	작성일	기술 스택	서비스 형태

최종 검토 결과

수정·삭제된 항목 (검토 완료)

항목	기존	수정 후	상태
서비스 유형	"오픈소스 AI 관측 플랫폼"	"AI 관측 플랫폼 (클라우드 SaaS)"	수정
서비스명	AgentOps (가칭만 표시)	AgentOps (가칭 · 정식명: Spanlens)	수정
초안/검증/2차 섹션	A~E 섹션 (설계 과정 전부)	전부 삭제 → 최종본만 유지	삭제
셀프호스팅 설명	"셀프호스팅 + 단순함 우선" (오픈소스처럼 읽힘)	"Docker 이미지로 셀프호스팅 지원 (소스코드 비공개)"로 명확화	수정
경쟁사 설명 표현	단순 "대안"	포지셔닝 명확히 — Helicone(인수됨), Langfuse(복잡함) 대안	유지
나머지 기술 내용	프록시 구조, 인증 계층, 보안 규칙 등	이상 없음 — 그대로 유지	유지

💡 **추가 발견: "오픈소스" 혼재 이유**

경쟁사(Helicone, Langfuse)가 오픈소스라서 기획서 작성 중 혼용됐습니다. 우리는 클라우드 SaaS + 선택적 셀프호스팅(Docker)으로 명확히 구분합니다.

최종 확정본

CLAUDE.md 파일 — 복사해서 프로젝트 루트에 저장

📋 **사용 방법**

아래 내용을 복사해 프로젝트 루트에

CLAUDE.md

파일로 저장하세요. Claude Code가 매 세션 자동으로 읽습니다.

```
# CLAUDE.md — AgentOps (가칭 · 정식명: Spanlens)

<!--
Claude Code 작업 지침서
리서치 기반: Helicone CLAUDE.md + Langfuse AGENTS.md + Anthropic Best Practices
3회 검증 완료 (2026.04)
-->

## 프로젝트

LLM 개발자를 위한 AI 관측 플랫폼 (클라우드 SaaS, 소스코드 비공개).
baseURL을 1줄 교체해 요청 로깅, 비용 추적, 에이전트 트레이싱 제공.
타깃: Helicone(인수됨)·Langfuse(복잡함)의 대안. Docker 이미지로 셀프호스팅 지원.
스택: Next.js 14 + Hono + Supabase PostgreSQL + TypeScript + pnpm monorepo

## 구조

apps/web/ — Next.js 14 대시보드 (App Router)
apps/server/ — Hono 서버 (LLM 프록시 + REST API 통합)
packages/sdk/ — JS/TS SDK (npm 배포용)
supabase/ — DB 마이그레이션(migrations/) + 시드(seeds/)

의존성 방향 (위반 금지):
apps/web → apps/server (fetch only, 직접 import 금지)
apps/server → supabase client
packages/sdk → 외부 패키지만 (apps/ 절대 import 금지)

핵심 데이터 흐름:
Client → POST /proxy/openai/v1/* → [API Key 검증] → [Provider Key 복호화] → OpenAI
응답 passthrough + tee() → 비동기 로깅 → requests 테이블

## 개발 명령어

### 로컬 시작
supabase start # 로컬 Supabase 실행 (Docker 필요)
supabase db push # 마이그레이션 적용
```

```
cp apps/server/.env.example apps/server/.env
pnpm install && pnpm dev # web:3000, server:3001

### 검증 — IMPORTANT: 코드 변경 후 반드시 실행
pnpm typecheck # TypeScript 타입 검사
pnpm lint # ESLint
pnpm test # 단위 테스트 (Vitest)
pnpm build # 최종 빌드 확인

### DB
supabase gen types --lang typescript --local > supabase/types.ts # 타입 재생성
supabase db reset # 로컬 DB 초기화 (주의: 전체 삭제)

## 변경 범위별 최소 검증

| 변경 범위 | 최소 검증 명령어 |
|---------------------|---------------------------------------------------|
| apps/web | pnpm --filter web typecheck && lint |
| apps/server | pnpm --filter server typecheck && lint && test |
| supabase/migrations | supabase db push && supabase gen types |
| packages/sdk | pnpm --filter sdk build && typecheck |
| 크로스 패키지 변경 | pnpm typecheck && pnpm lint (전체) |

## 인증 계층 — YOU MUST FOLLOW

/proxy/* 경로 → authApiKey 미들웨어 (API Key SHA-256 해시 검증)
/api/* 경로 → authJwt 미들웨어 (Supabase JWT)
DB 쓰기(로깅) → supabaseAdmin (service_role, RLS bypass)
DB 읽기(조회) → supabaseClient (anon key, RLS 적용)
두 미들웨어 절대 혼용 금지.

## 보안 규칙 — IMPORTANT (위반 시 보안 사고)

1. Provider Key(실제 OpenAI/Anthropic key) 절대 로그 출력 금지
2. Provider Key 복호화: apps/server/src/lib/crypto.ts의 aes256Decrypt()만 사용
3. 복호화 key는 fetch() Authorization 헤더에만 즉시 사용, 변수 저장 최소화
4. DB 저장 전 request_body에서 Authorization 헤더 제거 필수
5. 스트리밍: body.tee()로 복사, 원본 스트림 즉시 클라이언트 반환

## DB 작업 규칙

- 새 테이블 추가 시 반드시: ALTER TABLE t ENABLE ROW LEVEL SECURITY;
- 기존 마이그레이션 파일 수정 금지 → 새 파일 추가 (YYYYMMDDHHMMSS_desc.sql)
- supabase/types.ts 직접 수정 금지 → supabase gen types 사용
- 마이그레이션 실행 후 반드시 supabase gen types 재실행

## 핵심 모듈 — 중복 구현 금지

lib/crypto.ts — AES-256-GCM 암/복호화 (Provider Key 전용)
```

`lib/cost.ts` — 비용 계산 `calculateCost(provider, model, usage)`

`lib/logger.ts` — 비동기 로깅 `logRequestAsync(data)`

`lib/db.ts` — `supabaseAdmin` / `supabaseClient` 인스턴스

`parsers/openai.ts` — OpenAI 스트림 파서 (마지막 chunk에 usage)

`parsers/anthropic.ts` — Anthropic 파서 (**message_delta에 usage, OpenAI와 다름!**)

`parsers/gemini.ts` — Gemini 파서

코드 스타일

- Hono 에러 반환: `return c.json({ error: 'message' }, 401)`
- 비동기 로깅 fire-and-forget: `logRequestAsync(data).catch(console.error)`
- Tailwind만 사용 (inline style 금지)
- 서버 컴포넌트: 데이터 fetch / 클라이언트 컴포넌트: 인터랙션(useState, onClick)
- 새 패키지 추가: `pnpm add`만 사용 (npm/yarn 혼용 금지)

새 기능 추가 시 흐름

1. DB 변경 필요? → supabase/migrations/ 새 파일 → db push → gen types
2. API 엔드포인트 → apps/server/src/api/ 해당 라우터에 추가
3. 인증 미들웨어 → /api/* 는 authJwt, /proxy/* 는 authApiKey 반드시 확인
4. UI → apps/web에서 fetch('/api/v1/...') 또는 TanStack Query
5. 검증 → pnpm typecheck && lint && test

환경변수 (필수)

`.env.example` 참고. 핵심:

SUPABASE_URL, SUPABASE_ANON_KEY, SUPABASE_SERVICE_ROLE_KEY

ENCRYPTION_KEY=<32바이트 base64> ← **잘못 설정 시 Provider Key 복호화 조용히 실패**

PORT=3001 (server), 3000 (web)

Known Gotchas — AgentOps 특유의 함정

1. **스트리밍 토큰 0**: Anthropic usage는 message_delta에 있음 (OpenAI는 마지막 chunk). parsers/anthropic.ts 확인.
2. **비용 null**: model_prices에 모델 없으면 calculateCost()가 null 반환. 새 모델 추가 시 seeds/model_prices.sql 업데이트.
3. **RLS 차단**: anon 클라이언트로 INSERT → 403. 로깅은 반드시 supabaseAdmin 사용.
4. **spans FK 없음**: spans.parent_span_id는 FK 제약 없음 (의도적). 에이전트 병렬 span 지원. 직접 FK 추가 금지.
5. **복호화 빈 문자열**: ENCRYPTION_KEY 불일치 시 에러 대신 빈 문자열 반환 가능. 복호화 결과 항상 length 체크.

금지 사항

- git reset --hard 금지
- generated/ dist/ .next/ supabase/types.ts 직접 수정 금지
- 기존 supabase/migrations/*.sql 파일 수정 금지
- apps/web에서 Supabase 직접 접근 금지 (반드시 /api/ 경유)

```
- console.log에 key/secret/token 포함 금지
- pnpm 외 패키지 매니저 사용 금지
- lib/cost.ts, lib/crypto.ts 함수 다른 곳에 재구현 금지

## 커밋 규칙

Conventional Commits: type(scope): description
type: feat | fix | refactor | perf | test | docs | chore
scope: web | server | sdk | db | proxy
예: feat(proxy): add anthropic streaming support
```

AgentOps (가칭 · Spanlens) — CLAUDE.md v1.0 2026.04 · 최종 확정 Next.js 14 + Hono + Supabase